cp_11b

1

Secured URL | 20/20

Solved

Easy

Given an URL, write a program that checks if the given URL is a secured URL.

Print **True** if the given URL is a secured URL. Otherwise, print **False**.

Note

The url that starts with **https://** is a secured url

Input

The input will be a single line containing a string.

Output

The output should be a single line containing a boolean. **True** should be printed if the URL is secured. Otherwise, **False** should be printed.

Explanation

For example, if the given URL is **https://docs.google.com**,

- The given URL **https://docs.google.com** starts with **https://**.

The output should be **True**, as the URL is secured.

Sample Input 1

```
https://docs.google.com
```

Sample Output 1

```
True
```

code

```
url=input()
secured=url.startswith("https")
if secured:
    print("True")
```

```
else:
    print("False")
```

2

Python File | 25/25 — Solved

Easy

Given a file name with the file extension, write a program that checks if the given file is a Python file.

Print **True** if the file is a Python file. Otherwise, print **False**.

Note

- A file extension is added to the end of a file name to identify the file's format.
- A Python file has the extension **.py**.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing a boolean. **True** should be printed if the file is a Python file. Otherwise, **False** should be printed.

Explanation

For example, if the given file name is **add_numbers.py**,

- The given file name **add_numbers.py** ends with the extension **.py**.

The output should be **True**, as the given file is a Python file.

Sample Input 1

```
add_numbers.py
```

Sample Output 1

```
True
```

code

```
file=input()
extention=file.endswith(".py")
if extention:
    print("True")
```

```
else:
    print("False")
```

3

Palindrome | 25/25

Solved

Easy

Given a string, check if the given string is a palindrome.
A palindrome is a sequence of characters that can be read the same way whether you start from the beginning or the end.

Note

Consider uppercase characters and lowercase characters as different.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing a boolean. **True** should be printed if the given string is a palindrome. Otherwise, **False** should be printed.

Explanation

For example, if the given string is **madam**,

- The reverse of the string **madam** is **madam**.
- As the reversed string and the given string are the same, the string is a palindrome.

The output should be **True**, as the given string is a palindrome.

For example, if the given string is **Noon**,

- The reverse of the string **Noon** is **nooN**.
- Lowercase **n** is not the same as upper case **N**.
- As the reversed string and the given string are not the same, the string is not a palindrome.

The output should be **False**, as the given string is not a palindrome.

Sample Input 1

```
madam
```

Sample Output 1

```
True
```

code

```
string=input()
if string[0]==string[-1]:
    print("True")
else:
    print("False")
```

4

Palindrome - 2 | 25/25 ● Solved

Easy

You are given a string, write a program to find whether the string is palindrome or not.
Note: Treat uppercase letters and lowercase letters as same when comparing letters.

Input

The first line of input is a string.

Output

The output should be the string `True` or `False` .

Explanation

In the given example, the string `Madam` is a palindrome as we are treating `M` and `m` as equal. So, the output should be `True` .

Sample Input 1

```
Madam
```

Sample Output 1

```
True
```

Sample Input 2

```
Treat
```

Sample Output 2

```
False
```

code

```
string=input()
str_lower=string.lower()
rev_str=""
for i in range(len(string)):
```

```
    rev_str=str_lower[i]+rev_str
if rev_str==str_lower:
    print("True")
else:
    print("False")
```

5

Valid Password | 25/25 ● Solved

Given a password `S` , check if `S` is a valid password.
The password is valid only if it contains at least one digit.

Print **Valid Password** if `S` is a valid Password. Otherwise, print **Invalid Password**.

Input

The input will be a single line containing a string representing password `S` .

Output

The output should be a single line containing a string **Valid Password** if the password as at least one digit. Otherwise, **Invalid Password**.

Explanation

For example, if the given password is `S = Qwerty00` ,

- The characters in the password `S` are **Q**, **w**, **e**, **r**, **t**, **y**, **0** and **0**.
- The password **Qwerty00** contains **2** digits **0** and **0**.

The output should be **Valid Password**.

Sample Input 1

```
Qwerty00
```

Sample Output 1

```
Valid Password
```

code

```
password=input()
count=0
for i in password:
    char=i.isdigit()
    if char:
        count=count+1
```

```
if count>0:
    print("Valid Password")
else:
    print("Invalid Password")
```

6

Case Conversion | 35/35 ● Solved

Easy

Given a string, write a program to modifiy the string as given below.

- Add a space before each uppercase character excluding the first uppercase character.

Print the modified string.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing a string obtained by adding space before the uppercase characters.

Explanation

For example, if the given string is **TitleCase**,

- The uppercase characters in the given string **TitleCase** are **T** and **C**.
- The modified string after adding space before each uppercase character excluding the first uppercase character is **Title Case**.

The output should be **Title Case**.

Sample Input 1

```
TitleCase
```

Sample Output 1

```
Title Case
```

code

```
string=input()
new_str=string[0]
for i in range(1,len(string)):
    upper_char=string[i].upper()
    if string[i]==upper_char:
        new_str+=" "+string[i]
```

```
    else:
        new_str+=string[i]
print(new_str)
```

7

Case Conversion - 2 | 40/40

Solved

Easy

Given a string, write a program to modify the string as given below.

- Add a hyphen (-) before each upper-case character.
- Convert upper-case characters into lower-case characters.

Print the modified string.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing a string obtained by adding a hyphen (-) before each uppercase character and converting uppercase characters to lower-case characters.

Explanation

For example, if the given string is **aChristmasStory**,

- The upper-case characters in the string **aChristmasStory** are **C and S**.
- After adding a hyphen (-) before the upper case characters **C** and **S** the string is **a-Christmas-Story**.
- After converting the upper case characters **C** and **S** to the lower case characters **c** and **s** the string is **a-christmas-story**.
- The modified string after adding hyphen (-) before each uppercase character and convert uppercase characters to lower-case characters.

The output should be **a-christmas-story**.

Sample Input 1

```
aChristmasStory
```

Sample Output 1

```
a-christmas-story
```

code

```
string=input()
new_string=string[0]
```

```
for each_char in range(1,len(string)):
    upper=string[each_char].upper()
    lower=string[each_char].lower()
    if string[each_char]==upper:
        new_string=new_string+"-"+lower
    else:
        new_string=new_string+string[each_char]
print(new_string)
```

8

Valid Password - 2 | 20/20 ● Solved

Write a program to check whether the given password is valid or not.
Consider the password to be valid if it contains at least one uppercase letter.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing either "Valid Password" or "Invalid Password".

Explanation

For example, if the given password is "Qwerty00", the output should be "Valid Password" as it contains an uppercase letter.
Whereas, if the given password is "stadium", the output should be "Invalid Password" as it does not contain a uppercase letter.

Sample Input 1

```
Qwerty00
```

Sample Output 1

```
Valid Password
```

Sample Input 2

```
stadium
```

Sample Output 2

```
Invalid Password
```

code

```
password=input()
count=0
for i in password:
```

```
    if i.isupper():
        count+=1

if count>0:
    print("Valid Password")
elif count==0:
    print("Invalid Password")
```

9

Valid Password - 3 | 40/40 ● Solved

Write a program to check whether the given password is valid or not.
Consider the password to be valid if it contains at least one uppercase letter, one lowercase letter, and one digit.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing either "Valid Password" or "Invalid Password".

Explanation

For example, if the given password is "Qwerty00", the output should be "Valid Password" as it contains an uppercase letter, a lowercase letter, and a digit.

Whereas, if the given password is "Dashboard", the output should be "Invalid Password" as it does not contain a digit.

Sample Input 1

```
Qwerty00
```

Sample Output 1

```
Valid Password
```

Sample Input 2

```
Dashboard
```

Sample Output 2

```
Invalid Password
```

code

```
password=input()
is_digit=False
```

```
for each_ch  in password:
    if each_ch.isdigit():
        is_digit=True
is_upper=password.upper()==password
is_lower=password.lower()==password
is_upper_n_lower=(not is_upper) and (not is_lower)
is_valid=is_upper_n_lower and is_digit
if (is_valid):
    print("Valid Password")
else:
    print("Invalid Password")
```

10

Swap Case | 35/35 ● Solved

Given a word, write a program to convert all the uppercase letters to lowercase letters and vice versa.

Input

The input will be a single line containing a string.

Output

The output should be a single line containing the modified word.

Explanation

For example, if the given word is "Coding", your code should print the output by swapping the case of all letters ("cODING").

Sample Input 1

```
Coding
```

Sample Output 1

```
cODING
```

Sample Input 2

```
HUMANS
```

Sample Output 2

```
humans
```

code

```
string=input()
new_str=""
for each_ch in string:
    upper_ch=each_ch.isupper()
    lower_ch=each_ch.islower()
    if upper_ch:
        new_str+=each_ch.lower()+""
    else:
        new_str+=each_ch.upper()+""
print(new_str)
```